

PREMIER UNIVERSITY, CHATTOGRAM

B.Sc. in Computer Science and Engineering



Final Project Report

Course Name	Artificial Intelligence Lab
Course Code	CSE 3318
Name of Project Report Topic	AI-Based Sokoban Solver using DFS, BFS and UCS
Number of Project Report Topic	01
Date of Submission	18.04.2026

SUBMITTED BY		REMARKS :
THANJILUR RAHMAN SAYEED	IFTY ANWAR	
0222320005101162	0222320005101192	
SEMESTER: 5 TH	BATCH: 44	
SESSION: FALL – 25	SECTION: E	
GROUP NO. - 07	GROUP NAME – TEAM VISIONARY	

Contents

1. Problem Statement
2. Introduction
3. Objectives
4. Theoretical Background
 - 4.1 Depth First Search (DFS)
 - 4.2 Breadth First Search (BFS)
 - 4.3 Uniform Cost Search (UCS)
5. Modeling Sokoban Game
6. Methodology
7. Statistics and Evaluation
8. Description of the Flowchart
9. Results and Analysis
10. Screenshots with Figure & Description
11. Discussion & Conclusion
12. References

1. Problem Statement:

Sokoban is a puzzle-based game where a player must push boxes to target locations. Finding the correct sequence of moves is difficult due to obstacles and movement limitations.

The objective of this project is to apply Artificial Intelligence search algorithms such as DFS, BFS, and UCS to solve the Sokoban game and evaluate their performance based on efficiency, solution length, and memory usage.

2. Introduction:

Artificial Intelligence (AI) search algorithms are widely used to solve complex computational problems by exploring possible states. In this project, the Sokoban game is modeled as a search problem. Each state represents the position of the player and boxes. The goal is to move all boxes to their designated target positions. We implement DFS, BFS and UCS algorithms and compare their effectiveness using experimental analysis.

3. Objectives:

- To model Sokoban as a search problem
- To implement DFS, BFS and UCS algorithms
- To analyze solution length and node expansion
- To compare algorithm performance
- To determine the most efficient algorithm

4. Theoretical Background:

4.1 Depth First Search (DFS)

DFS explores one path deeply before backtracking. It is memory efficient but does not guarantee an optimal solution.

4.2 Breadth First Search (BFS)

BFS explores nodes level by level. It guarantees the shortest path but requires high memory.

4.3 Uniform Cost Search (UCS)

UCS selects the node with the lowest cost first. It guarantees an optimal solution and performs efficiently.

5. Modeling Sokoban Game:

Solving a Sokoban task can be modeling as a search problem which we need to find a solution (path = sequence of actions including 'up', 'down', 'left', 'right') so that from initialize state of game we can go to the end-state that wins the game.

Thus, we can modeling the Sokoban World by some concepts such as:

- a) State: information about position of the player and position of boxes.
- b) Start state: the state when we perform one of three functions (DFS, BFS, UCS). Typically, it is when we start a level of game.
- c) End state: the state where we win the game (all boxes are in the correct position).
- d) State Space: all states that can be reached from the start state. It is the totality of possible positions of player and boxes. It is frequently represented as a graph.
- e) Action: 'up', 'down', 'left', 'right' correspond to moving the player up, down, left and right.
- f) Legal Action: an action is called 'Legal' if it makes the function `solver.py/isLegalAction` true. In other words, it does not cause the player to move into walls or boxes. Beside, in algorithms, an action only be considered if it is not Failed. It does not cause a box that is not in the correct position but it can not move anymore.
- g) Successor function: in this setting code, it is function `solver.py/legalAction`. A successor function describes the agent's options: given a state, it returns a set of (action, next state) pairs. it indicates actions and states corresponding can happen to an input state.

6. Methodology:

- a) Input Sokoban level
- b) Apply DFS, BFS, UCS
- c) Generate solution paths
- d) Measure:
 - a. Solution length (L)
 - b. Number of nodes (N)
- e) Compare results

7. Statistics and Evaluation

Let's evaluate the performance of algorithms, we need to measure some quantity such as: length of solution (number of actions that need to be performed) - L, number of nodes (state) that can be processed by algorithm - N.

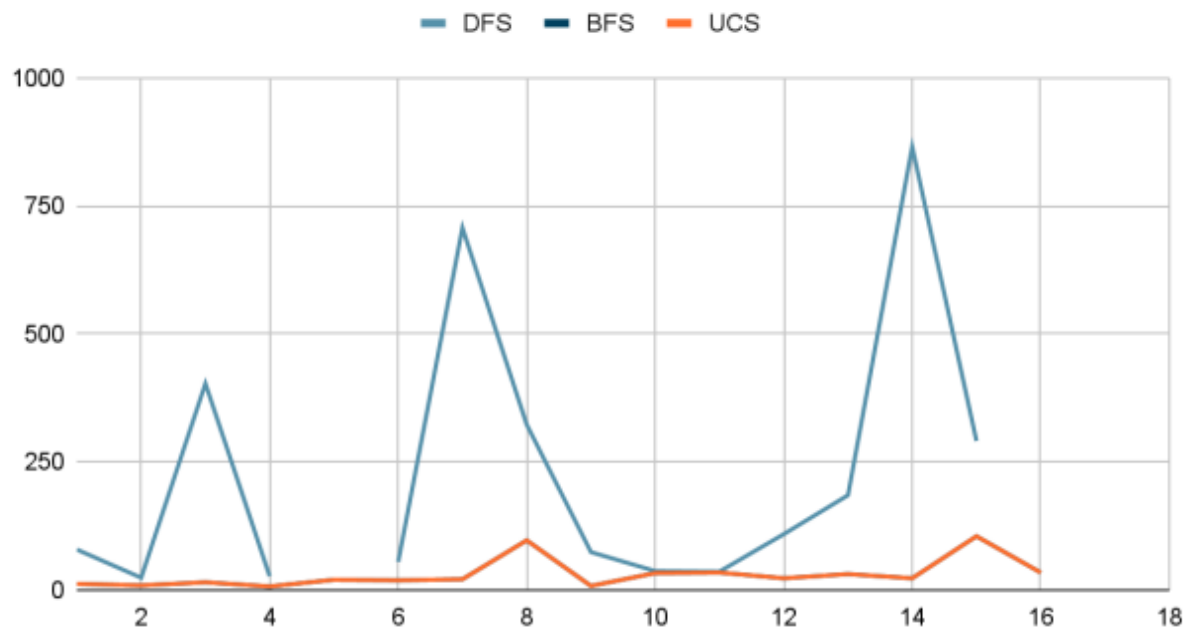
Statistics Algorithms Table - 1

Level of Game	DFS (L)	DFS (N)	BFS (L)	BFS (N)	UCS (L)	UCS (N)
1	79	2177	12	2552	12	1634
2	24	<u>55</u>	9	186	9	181
3	403	4582	15	2137	15	1218
4	27	<u>41</u>	7	230	7	171
5	OOM	–	20	2253288	20	1278998
6	55	<u>524</u>	19	612	19	595
7	707	<u>16221</u>	21	27307	21	16767
8	323	<u>1756</u>	97	5561	97	5356
9	74	7142	8	141	8	174
10	37	<u>392</u>	33	509	33	524
11	36	<u>638</u>	34	671	34	657
12	109	5116	23	2959	23	2966
13	185	6341	31	4905	31	6011
14	865	88800	23	65221	23	69842
15	291	<u>3842</u>	105	6362	105	6325
16		OOM	34	160839	34	141814
17	x	169158	x	169158	x	169158
18		OOM		OOM		OOM

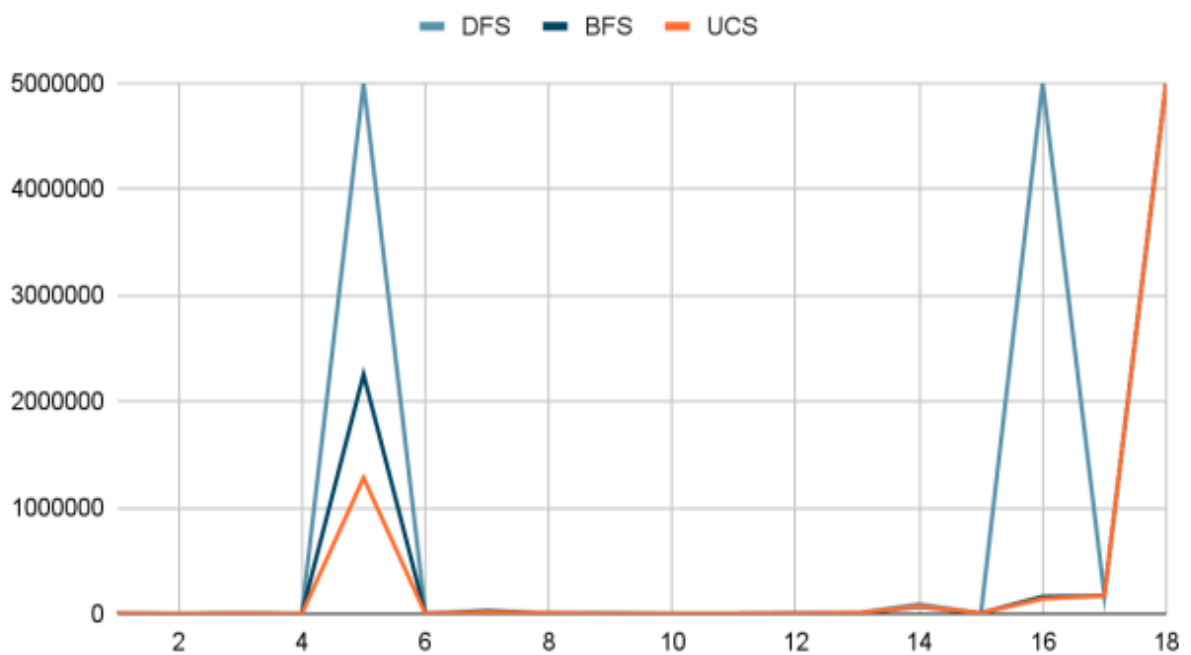
Notion:

- [x]: not find a solution
- [OOM]: Out of Memory
- Computer used for statistics is Kaggle Colab P100: RAM 29GB.

Number of actions (Length)



Number of nodes



A. Evaluate about Length (number of actions)

- In these maps of the Sokoban Game, both BFS and UCS find the shortest path (minimum number of actions) if there is a solution. But typically, UCS will search for the solution in a shorter time, because it processes fewer nodes.
- DFS does not find a shortest path, the path can be very long. But, in some maps that have many solutions to win and length of solution is not deep, it can be effective.

B. Evaluate about Performance of Algorithms

- We expect an algorithm to find the exact solution (shortest path) and the execution time is minimal. The accuracy of the solution is expressed by the length of the solution and the execution time is expressed by the number of nodes that can be processed by the algorithm.
- We define a quantity as the goodness of the algorithm as follows: $G = L + 2N$
- Because, we expect to find the solution faster, so the weight of N will be larger than L.
- But L and N are two different quantities (they are not in the same range) so we need to normalize them:
 - $L = (L - 7)/(865 - 7)$
 - $N = (N - 41)/(2253288 - 41)$
 - If it OOM, $G = 3$.

Comparison Table - 2

Level	DFS	BFS	UCS	Total G
1	0.085812015	0.008056289	0.005791188	0.099659492
2	0.019825946	0.002459705	0.002455267	0.024740919
3	0.465569089	0.011184436	0.010368724	0.487122249
4	0.023310023	0.000167758	0.000115389	0.02359317
5	3	2.015151515	1.150363944	6.165515459
6	0.056372771	0.014492838	0.014477749	0.085343358
7	0.830212313	0.040518535	0.031163147	0.901893995
8	0.369820616	0.109794701	0.109612741	0.589228058
9	0.084391482	0.001254262	0.001283553	0.086929297
10	3	0.030718431	0.030731745	3.061450176
11	0.034329436	0.032027724	0.032015298	0.098372458
12	0.123385729	0.021238059	0.021244272	0.16586806
13	0.213051138	0.032289353	0.033271048	0.278611538
14	1.078783196	0.076502306	0.080603943	1.235889445
15	3	0.119829684	0.119796843	3.239626527
16	3	0.174194118	0.157307376	3.331501495
17	1.150109597	1.150109597	1.150109597	3.450328792
18	3	3	3	9

Total G Values:

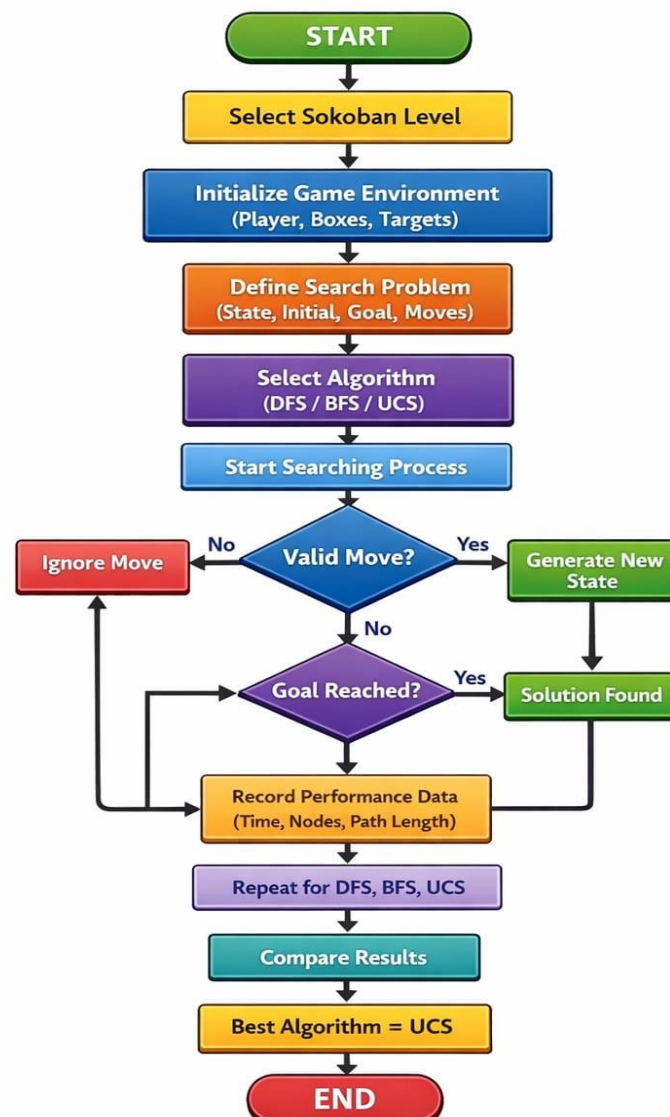
- **DFS Total G:** 19.53497335
- **BFS Total G:** 6.839989313
- **UCS Total G:** 5.950711825

If G is larger, it shows that the algorithm is not good, so the best algorithm is the algorithm with the smallest total G. In other words, UCS is the best algorithm in these cases.

C. Evaluate about Game Difficulty

- The map that is most difficult for a computer to solve will be the one that takes the longest to solve.
- The larger the number of nodes an algorithm needs to process, the longer it will take to run the algorithm.
- So, following figure 2, level 18 is most difficult. It causes all 3 algorithms to suffer Out of Memory (29 GB).
- But, if we ignore the OOM case, maybe that level 5 is most difficult.

8. Description of the Flowchart



1. Start

- The process begins with the start of the Sokoban problem-solving system.

2. Select Sokoban Level

- A specific Sokoban level is selected.
- Each level has a different layout of walls, boxes, and target positions.

3. Initialize Game Environment

- The system initializes all game elements:
 - Player position
 - Box positions
 - Target locations
- This forms the initial configuration of the game.

4. Define Search Problem

- The problem is formally defined using:
 - **State:** Current positions of player and boxes
 - **Initial State:** Starting configuration
 - **Goal State:** All boxes placed on target locations
 - **Actions:** Possible moves (up, down, left, right)

5. Select Search Algorithm

- One search algorithm is selected from:
 - Depth-First Search (DFS)
 - Breadth-First Search (BFS)
 - Uniform Cost Search (UCS)

6. Start Searching Process

- The algorithm begins exploring possible moves from the initial state.

7. Check Valid Move

- Each possible move is checked:
 - If **invalid** (wall collision or illegal push) → move is ignored
 - If **valid** → proceed to next step

8. Generate New State

- A new state is created based on the valid move.

- The updated positions of player and boxes are recorded.

9. Check Goal State

- The system checks whether:
 - All boxes are placed on target positions
- If **yes** → solution found
- If **no** → continue searching

10. Continue Searching (Loop)

- The algorithm keeps exploring new states.
- This process continues until a solution is found.

11. Solution Found

- Once the goal state is reached:
 - The solution path is generated
 - Steps required to solve the puzzle are recorded

12. Record Performance Data

- The system stores important metrics:
 - Execution time
 - Number of nodes expanded
 - Path length (number of moves)

13. Repeat for Other Algorithms

- The same process is repeated for:
 - DFS
 - BFS
 - UCS

14. Compare Results

- Performance of all algorithms is compared.
- Efficiency is evaluated based on time, cost, and optimality.

15. Best Algorithm Selected

- Uniform Cost Search (UCS) is identified as the best:
 - Provides optimal solution
 - Ensures minimum cost path

16. End

- The process ends after final comparison and result analysis.

9. Results and Analysis

Observations:

- BFS and UCS find shortest path
- UCS explores fewer nodes than BFS
- DFS produces longer paths
- Some levels cause Out of Memory (OOM)

Performance Formula:

$G = L + 2N$; Lower G value indicates better performance.

➡ Experimental results show: UCS has lowest G → best performance

10. Screenshots with Figure & Description

Figure 1: Sokoban Game Start Menu

Description: This screenshot shows the main menu of the Sokoban game where the user can start a new game, continue a previous game or quit the application.



Figure 2: Sokoban Game Environment (Initial State)

Description: This figure shows the initial configuration of the Sokoban level including the player, boxes, walls and target positions.

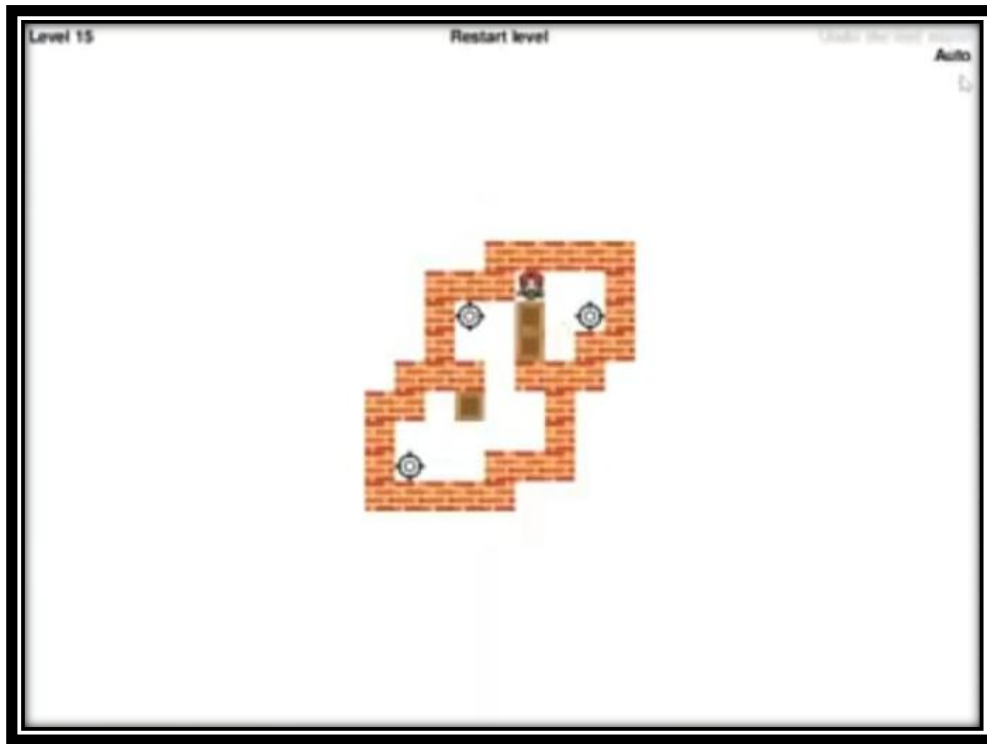


Figure 3: Player Movement and Box Interaction

Description: This screenshot demonstrates how the player moves within the grid and pushes boxes toward target locations.

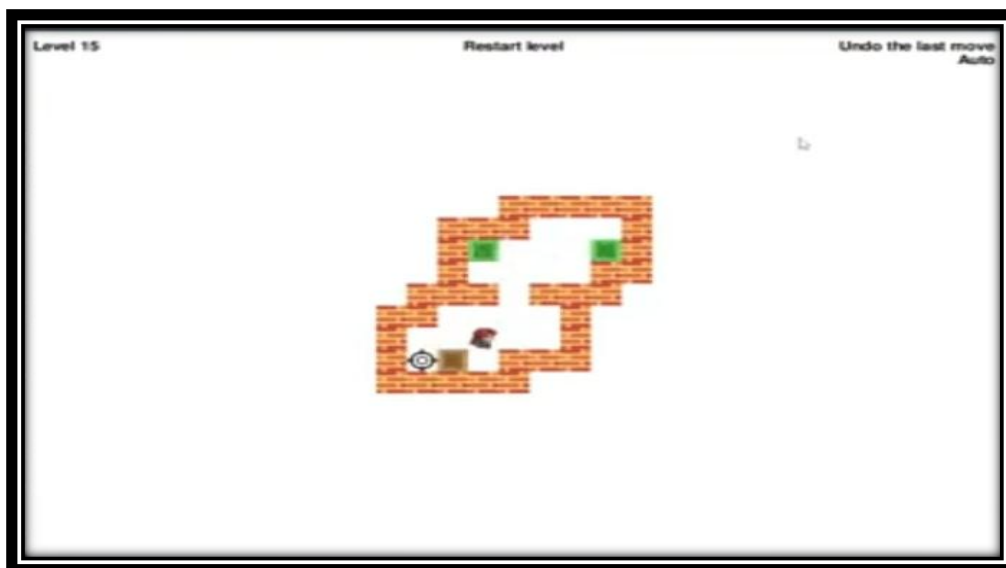


Figure 4: Algorithm Execution (Auto Solve)

Description: This figure shows the execution of the selected search algorithm where the system automatically explores different states to find a solution.

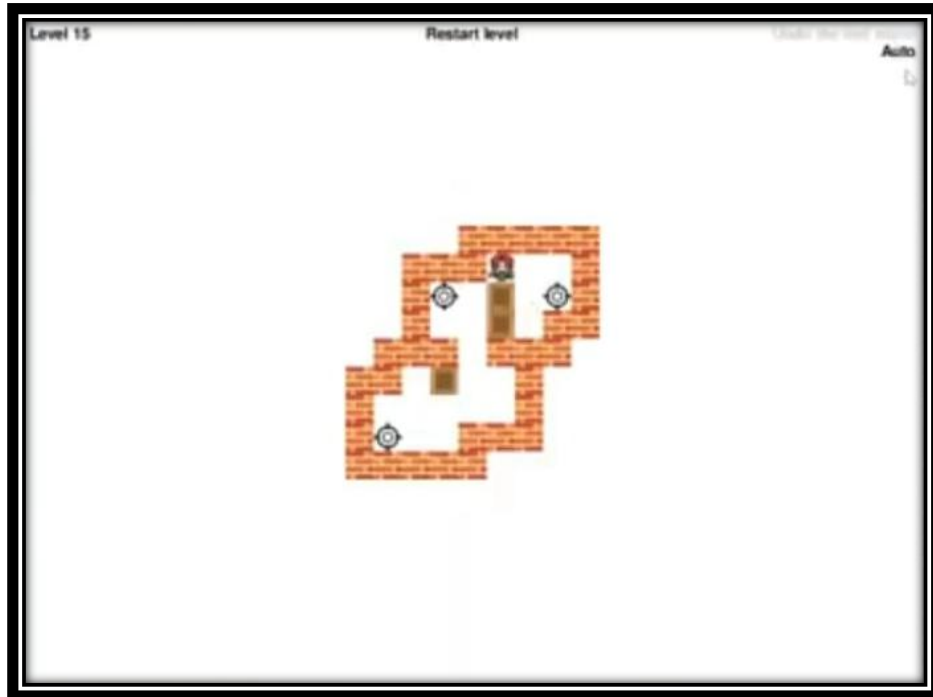
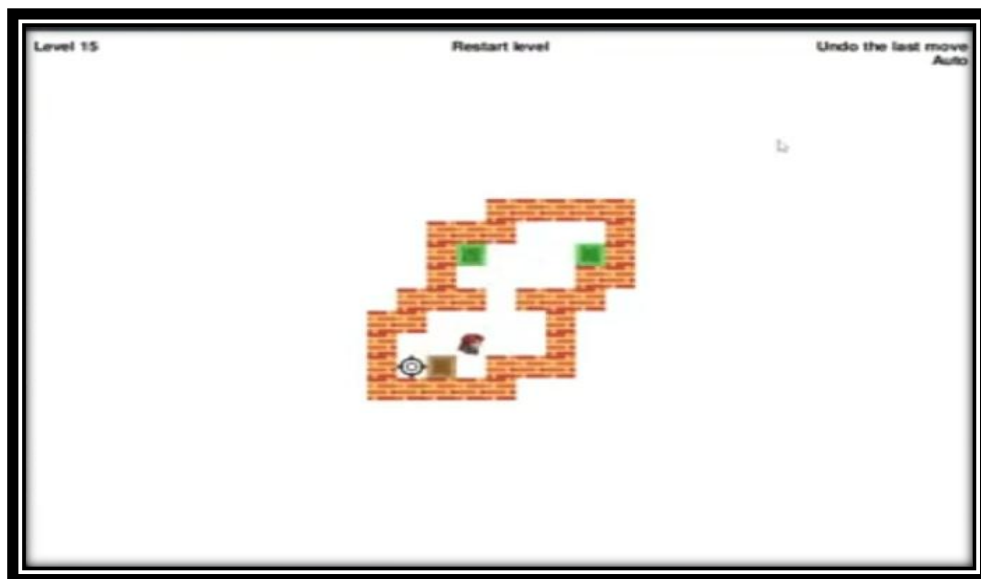


Figure 5: Goal State Achieved

Description: This screenshot shows the final state of the game where all boxes are successfully placed on their respective target positions.



11. Conclusion & Discussion

We can model Sokoban World by a search problem and apply algorithms such as DFS, BFS, UCS to search for a solution (a path) that can win the game. In these cases, UCS appears to be the better algorithm. There are many different ways to define the difficulty of the game, but if measured in terms of time, it seems that levels 18 and 5 are the most difficult. In the source code of this project, we use Python Programming Language to implement the algorithms, but the run time of programs is too long and slow (up hours). Instead of, you can use C++ as the backend of algorithms, it will significantly improve speed of algorithms based on data structure and control structure, thus reducing the time and resources needed to experiment.

12. References

- Quan Hoang Ngoc, *Application of DFS, BFS, UCS for Sokoban Game*
- GitHub Repository:
https://github.com/QuanHoangNgoc/Application_DFS_BFS_UCS_for_Sokoban_Game
- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3rd Edition
- GeeksforGeeks, *Search Algorithms in AI*
<https://www.geeksforgeeks.org/search-algorithms-in-ai/>
- Wikipedia, *Breadth-First Search, Depth-First Search, Uniform Cost Search*
<https://en.wikipedia.org>